EdgeSafe

An Edge-Based Weapon Detection and Secure Event Logging System for Educational Environments

1. Introduction

Firearm-related incidents in public and semi-public spaces have intensified over the past decade, with mass shootings in the United States becoming increasingly lethal and producing substantial public health consequences. Although educational institutions represent a smaller fraction of total incidents, attacks in schools and universities cause disproportionate psychological harm, including persistent anxiety, post-traumatic stress symptoms, and long-term erosion of perceived safety across academic communities. Epidemiological analyses show that once a firearm is introduced into a populated environment, casualty counts escalate rapidly, making early situational awareness and immediate response critical [1]. Firearms have recently become the leading cause of death among young people in the United States, surpassing other causes of mortality, including motor vehicle crashes [2]. This alarming shift underscores the growing vulnerability of children and adolescents and highlights the urgent need for preventive and early detection mechanisms in environments where young people live and learn. Traditional security infrastructures in educational facilities, such as surveillance cameras, access control systems, and human monitoring, are largely reactive. These approaches depend heavily on continuous human supervision, which is susceptible to fatigue, delayed reaction, and limited situational awareness. As a result, potentially dangerous situations may go unnoticed until escalation occurs.

Recent advances in computer vision and deep learning have enabled automated visual analysis for security applications. Object detection architectures such as YOLO have demonstrated strong performance in real-time firearm detection tasks, providing the technical foundation for proactive monitoring systems [3]. However, many existing commercial and research solutions rely on cloud-based platforms, including Amazon Rekognition and Microsoft Azure Video Analyzer, which require transmitting visual data to remote servers for processing. While scalable, cloud-centric approaches introduce latency, depend on stable connectivity, and raise significant privacy concerns in sensitive environments such as schools. Edge computing offers an alternative by enabling local inference on on-site devices, thereby reducing latency and limiting exposure of sensitive visual data [4]. Research has demonstrated that automatic handgun detection using deep learning is technically feasible under diverse real-world conditions, including variations in lighting, scale, and orientation [6]. However, prior work has largely emphasized model accuracy under controlled settings, without fully addressing deployment constraints such as on-device latency, privacy preservation, and secure event traceability. This reveals a critical gap between algorithmic capability and responsible, real-world system integration. Beyond detection accuracy, modern security systems must ensure accountability. Secure logging of detection events, preservation of evidentiary artifacts, and controlled access to historical records are essential for post-incident auditing and institutional oversight. Prior research highlights the importance of

traceability and controlled access mechanisms in AI-based surveillance systems, particularly in environments where the integrity and authenticity of logged events must be protected [5].

Motivated by these challenges, this project proposes an edge-based firearm detection system tailored for educational environments. The system is implemented as a web-enabled edge device that hosts both the machine learning inference engine and a secure dashboard interface. The detection pipeline executes autonomously over controlled input images, with results (including confidence scores and structured metadata) stored locally and made accessible to authenticated administrative users. As a stretch goal, the architecture will be extended to support real-time image capture through a connected camera, enabling continuous monitoring using the same secure processing and logging framework. By integrating edge AI, secure access control, and auditable event management, the proposed system aims to bridge the gap between technical feasibility and responsible deployment in safety-critical environments.

2. Problem statement

Educational institutions face significant challenges in preventing and responding to firearm-related security threats within their facilities. Existing security infrastructures are largely reactive, relying on human monitoring of surveillance feeds or post-event analysis rather than proactive threat detection. This creates a critical gap between the moment a potential threat appears and the time at which appropriate action must be taken, particularly in environments where rapid response is essential to minimize harm. From a technical perspective, current automated weapon detection solutions frequently depend on cloud-based machine learning services, which introduce latency, require continuous network connectivity, and raise concerns regarding the handling of sensitive visual data. These limitations reduce system reliability in safety-critical environments and make such solutions less suitable for deployment in educational settings, where privacy, low latency, and operational robustness are primary requirements.

The target domain of this project is physical security within educational environments, with a focus on schools and similar controlled-access facilities. The primary stakeholders include: school administrators, campus security personnel, and institutional IT departments responsible for maintaining secure and reliable infrastructure. Secondary stakeholders include: students and faculty, who benefit indirectly through improved situational awareness and enhanced safety without direct interaction with the system.

The scope of the proposed solution is intentionally constrained to support development within a single academic semester. The system focuses on detecting the presence of firearms through automated analysis of controlled image inputs rather than performing large-scale behavioral prediction or fully distributed surveillance. The solution is implemented as a web-enabled edge device that autonomously executes the detection pipeline and publishes results to a secure dashboard accessible to authorized users. Additionally, the system does not attempt to predict intent or automatically trigger enforcement actions;

instead, it is designed to provide timely, structured, and auditable information to assist human decision-makers.

Although deep learning based weapon detection models already exist and have demonstrated good performance in identifying firearms in visual data, their effective deployment in educational settings or environments remains a largely unsolved problem. Most already available solutions focus and emphasize the use of model accuracy in isolation, while not taking into account the low latency on-device interference and, privacy preservation, reliable operation under limited connectivity, and secure handling of detection of events. As a result of this, institutions are faced with a gap between the availability of weapon detection algorithms and the practical ability to deploy them responsibly within real world security infrastructures. This gap means that it is required to address not only the creation of brand new detection models, but also the design of an integrated system architecture that enables everything to operate reliably and in a secure way. The proposed solution lies in the integration of edge-based machine learning inference with secure, auditable event logging in a lightweight and privacy-preserving architecture. By performing weapon detection directly on a low-power edge device and limiting data transmission to authenticated local communication, the system reduces latency and minimizes exposure of sensitive data. Furthermore, the inclusion of structured event logs with associated metadata supports traceability and accountability, enabling post-event analysis and continuous evaluation of security procedures. This combination of edge AI, secure access control, and auditable logging distinguishes the proposed system from purely model-centric or cloud-dependent approaches, making it well-suited for a Capstone-level engineering project.

3. Objectives

The primary objective of this Capstone project is to design and implement an edge-based firearm detection system that supports proactive security monitoring in educational environments while addressing constraints related to latency, privacy, and system accountability. To achieve this goal, the project defines the following SMART objectives:

1. **Edge-Based Detection Performance:**
   Implement a deep learning–based weapon detection model deployed on a low-power edge device capable of processing input images with an end-to-end inference latency of ≤ 1 second per image and achieving a detection accuracy of at least 80% on a test dataset representative of the target domain.
2. **Autonomous Detection Pipeline:**
   Develop an automated detection workflow in which the edge device executes the inference process on preloaded image inputs without manual triggering, publishing detection results to a locally hosted web dashboard in real time.
3. **Event Logging and Auditability:**
   Design and implement a local event logging mechanism that records detected high-risk events with associated metadata, including timestamp, confidence score, and

image reference. The system shall correctly store and retrieve 100% of logged events through authenticated access, supporting post-event review and traceability.

4. **System Usability and Reliability:**
   Develop a functional end-to-end prototype that allows authorized users to receive detection feedback, and review historical events through a web interface, demonstrating stable operation during a controlled demo session without system crashes or data loss.
5. **Secure Web Dashboard and Access Control**
   Implement a web-based dashboard hosted on the edge device that allows authenticated users to securely access detection results and historical logs, ensuring that 100% of dashboard access requests require authentication and are protected via encrypted communication (HTTPS).

4. Solution Approach

4.1 Proposed Solution Overview

The proposed solution, *EdgeSafe*, is an edge-based firearm detection and secure event logging system designed for educational environments. The system integrates a deep learning object detection model with local, on-device inference to enable real-time identification of firearms while minimizing latency and preserving data privacy. The architecture is implemented as a web-enabled edge device that hosts both the machine learning inference engine and a secure web dashboard. The detection pipeline executes autonomously on the edge device using preloaded image inputs for proof-of-concept evaluation. Upon detection of a firearm, the system generates a structured alert containing relevant metadata, including the detection timestamp, model confidence score, and an isolated image of the detected object. These events are recorded in a secure local database and made accessible through an authenticated web-based dashboard. All communication between system components is encrypted, ensuring confidentiality and integrity of sensitive visual data. The solution is designed as a proof of concept, with an architecture that can be extended to support continuous video streams and multiple camera inputs in future deployments. As a stretch goal, the architecture will be extended to support continuous image capture from a connected camera, enabling automated real-time detection and publication of results through the same secure logging and visualization pipeline.

4.2 Gap with Existing Solutions

Existing automated weapon detection and video surveillance systems predominantly rely on cloud-based or cloud-dependent architectures. In such systems, captured images or video streams must be transmitted to remote servers for processing and analysis. While this approach benefits from scalable computational resources, it introduces several limitations that make it unsuitable for safety-critical educational environments.

First, cloud-based solutions introduce additional latency due to network transmission and remote inference, which can delay threat recognition during time-sensitive situations. Second, they depend on stable and continuous internet connectivity, reducing reliability in environments with limited or disrupted network access. Third, transmitting and storing sensitive visual data off-site raises significant privacy and data governance concerns, particularly when monitoring minors and controlled-access facilities. Finally, many existing solutions emphasize detection accuracy in isolation, without adequately addressing deployment constraints such as privacy preservation, secure event auditing, and local system resilience. This creates a gap between the availability of effective weapon detection algorithms and the ability to deploy them responsibly and reliably in real-world educational settings.

4.3 Value Proposition

The primary value proposition of *EdgeSafe* lies in its fully local, edge-based architecture combined with secure and auditable system design. By performing firearm detection directly on an edge device, the system significantly reduces detection latency and eliminates dependence on cloud infrastructure. This enables faster situational awareness and response while maintaining operational reliability under limited connectivity conditions. Additionally, the system is designed with security and privacy as core requirements. All data transmission is encrypted, access to the system is authenticated, and sensitive visual data remains within the local environment. The inclusion of structured event logging further supports accountability, post-event analysis, and continuous improvement of security procedures. Together, these features distinguish the proposed solution from cloud-centric or model-only approaches and make it well suited for deployment in educational environments.

4.4 Benefits and Limitations

4.4.1 Benefits to the Target Audience

For school administrators, campus security personnel, and IT departments, the proposed solution offers improved security responsiveness and reduced operational risk. Real-time, on-device detection enables rapid identification of high-risk situations, while local processing minimizes exposure to privacy breaches and cyberattacks. Students and faculty benefit indirectly through enhanced safety and situational awareness without requiring direct interaction with the system.

4.4.2 Limitations of the Solution

The scope of the system is intentionally constrained to support development within a single academic semester. The proof-of-concept implementation focuses on automated processing of controlled image inputs rather than continuous video stream analysis. The stretch goal of integrating a live camera feed may not be fully realized within the project timeline. Additionally, the system does not attempt to infer intent or automatically initiate

enforcement actions; it is designed solely to support human decision-making through timely and reliable information.

4.5 Required Resources

The implementation of *EdgeSafe* requires the following resources:

- A pre-trained deep learning object detection model (e.g., YOLO) adapted for firearm detection
- A low-power edge computing platform such as a Raspberry Pi or NVIDIA Jetson Nano
- A local web server framework to host the dashboard and API
- Firearm image datasets for testing and validation
- A web-based dashboard for authenticated system interaction and event review
- A controlled testing setup, using non-functional or simulated firearm images strictly for proof-of-concept evaluation
- (Stretch goal) A compatible camera module or USB webcam for real-time input

4.6 Intellectual Property Considerations

The proposed project leverages existing open-source technologies and publicly available research, including deep learning architectures and edge computing frameworks. No proprietary datasets or licensed commercial models are used in the development of the system. All software components developed as part of this project, including system integration code, deployment configurations, and secure event logging mechanisms, constitute original work created by the project team. Any intellectual property generated by this project is intended for academic and educational purposes. The system design, implementation, and documentation may be shared for research, instructional, or non-commercial use, subject to applicable open-source licenses and institutional guidelines. Furthermore, any intellectual property developed as part of this project will comply with and be managed in accordance with the intellectual property policies and regulations of the University of Puerto Rico at Mayagüez, if applicable.

5. Technical Approach

5.1 System Architecture Framework

EdgeSafe is designed using an edge-first, client–server architectural framework in which a single edge device functions as both the application server and the machine learning inference node. This architectural choice was made to minimize end-to-end latency, reduce dependency on external cloud services, and preserve sensitive visual data within a controlled local environment, critical requirements for security systems deployed in educational settings. The system follows a modular, service-oriented design that separates responsibilities across

core components (web interface, inference pipeline, and event logging). This separation of concerns improves maintainability, enhances system reliability, and allows independent modification of components, such as replacing the input module, without requiring structural redesign. At the same time, the architecture remains lightweight enough to operate efficiently on low-power hardware, aligning with the project's resource constraints.

At a high level, the architecture consists of:

- **Edge Device**: Hosts the web server, authentication layer, machine learning inference runtime, and local database. It is responsible for executing the detection pipeline autonomously, generating detection results, and persisting structured event logs.
- **Web Client**: A standard web browser that connects to the edge device to authenticate and view the dashboard. The client does not perform inference; it only requests and visualizes stored results and system status.

The system is implemented as a three-layer architecture:

1. **Presentation Layer** (Dashboard UI): A web-based dashboard served directly by the edge device. It provides authenticated access to detection results, event history, and basic system status while ensuring controlled exposure of sensitive information.
2. **Application Layer** (API + Pipeline Orchestrator): A lightweight backend service responsible for coordinating automatic execution of the inference pipeline over controlled input images and exposing authenticated endpoints for secure data retrieval. This layer encapsulates business logic and enforces access control mechanisms.
3. **Data Layer** (Local Storage): A local database (SQLite) stores structured event records, including timestamps, confidence scores, detection summaries, and image references. Associated image artifacts are maintained in local storage to preserve evidentiary integrity and support auditability.

This layered framework was selected to support low-latency processing, privacy preservation, operational resilience under limited connectivity, and incremental extensibility. As a stretch goal, the input module can be extended from preloaded images to a live camera feed without altering the presentation, logging, or authentication subsystems. This demonstrates architectural stability while enabling scalable evolution toward continuous monitoring deployments.

Figure 1 illustrates the operational pipeline of the proposed EdgeSafe architecture. The diagram depicts the flow of data from image acquisition through edge-based inference, event generation, secure storage, and dashboard visualization. This operational view complements the architectural framework described previously by detailing how system components interact during runtime.

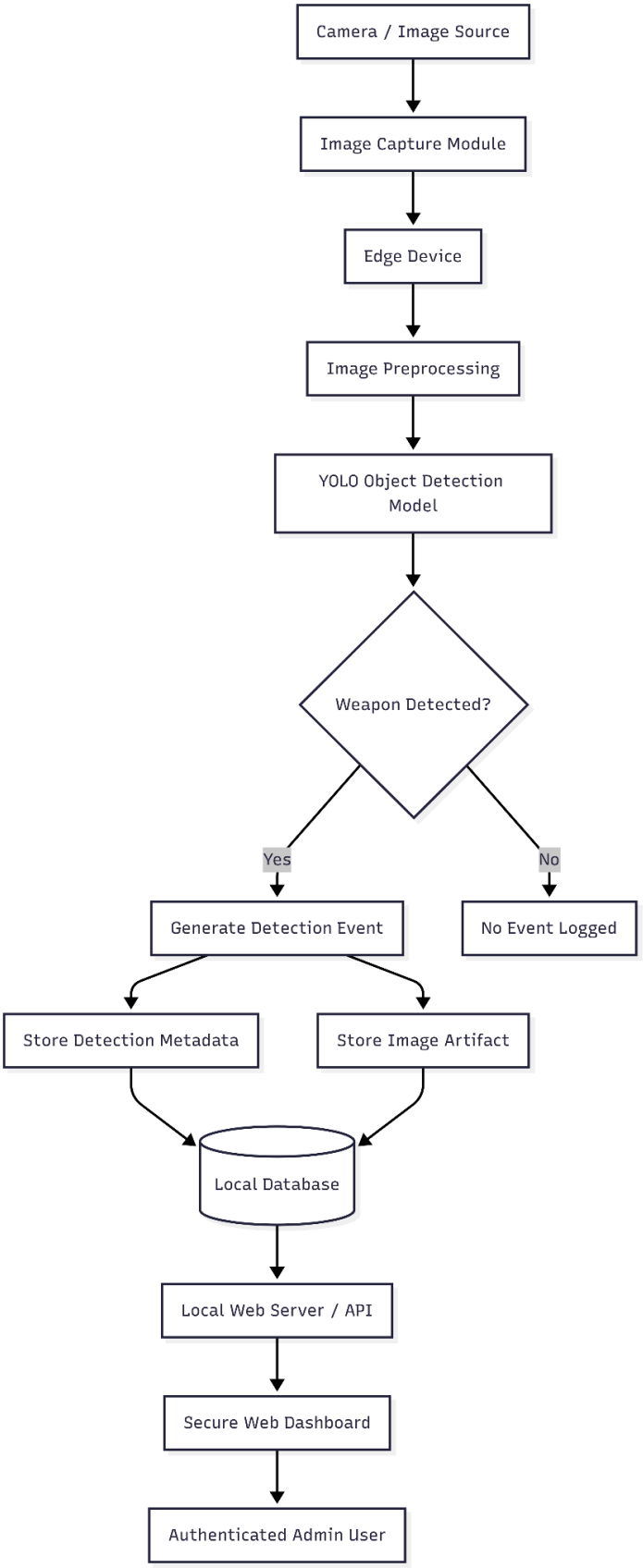

Figure 1. *EdgeSafe System Operational Pipeline*

As shown in Figure 1, the pipeline begins with an image source, which may consist of preloaded test images in the proof-of-concept stage or a live camera feed in the extended deployment scenario. The image capture module forwards input data to the edge device, where preprocessing is performed prior to inference.The YOLO-based object detection model analyzes the processed image and determines whether a firearm is present. If a weapon is detected, a structured detection event is generated, including metadata such as timestamp and confidence score. Both the metadata and associated image artifact are stored locally within the database layer. If no weapon is detected, no event is logged.Logged events are exposed through the local web server API and visualized in the secure dashboard, accessible only to authenticated administrative users.

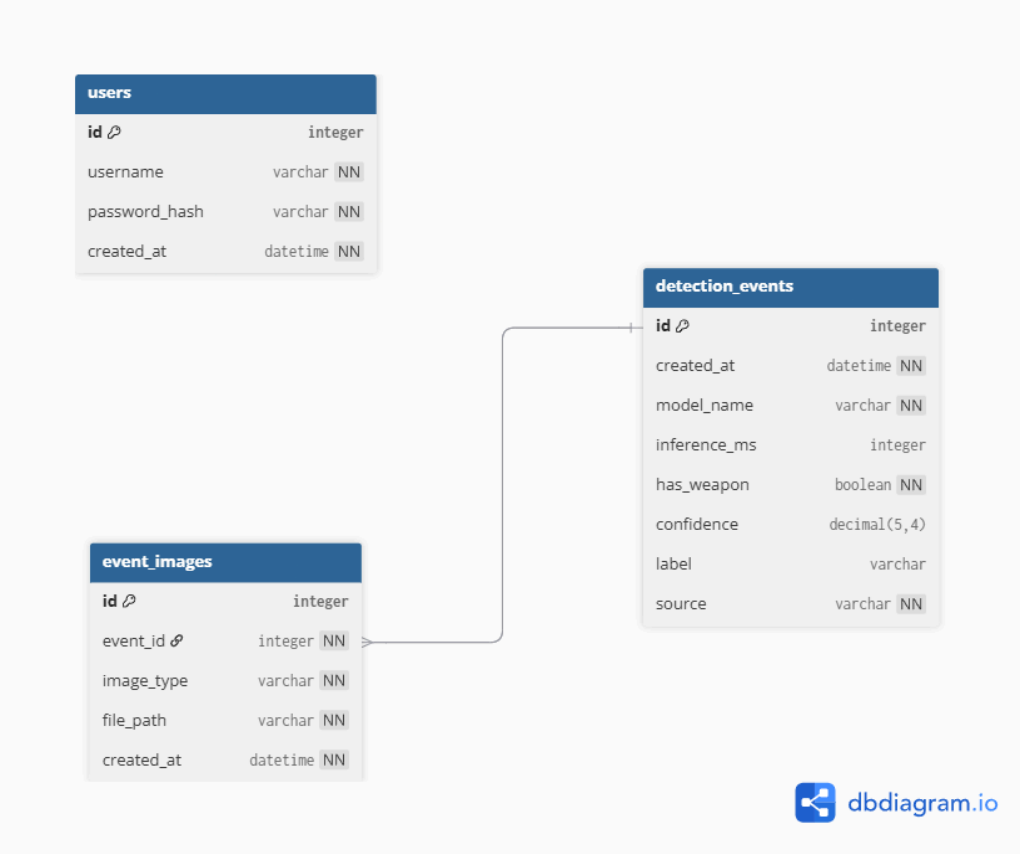

Figure 2. *EdgeSafe Event Logging Database Schema*

Figure 2 presents the relational database schema used to support secure event logging and auditability within the EdgeSafe system. The schema is structured around the detection_events table, which stores metadata generated during each inference execution, including timestamp, model identifier, inference latency, detection outcome, and confidence score. The event_images table maintains references to associated image artifacts linked through a foreign key relationship (event_id), preserving evidentiary integrity while separating metadata from file storage. This design improves modularity and supports efficient querying of detection records. The users table enables authenticated access control, ensuring that only authorized administrative users can retrieve stored events and image artifacts through the dashboard interface. This schema was designed to maintain data integrity, enforce

relational consistency, and support traceability of all detection events, aligning with the project's security and accountability objectives.

5.2 Related Works

The development of the system will build upon existing machine learning models, libraries, and software tools that are already available rather than developing all of the project components from scratch. It will integrate tools for object detection, web based design, secure communication, and structured data storage to construct a deployable edge based firearm detection system.

5.2.1 Pretrained Object Detection Models

The system will utilize a pretrained YOLO model made for object detection [3], in this case, specifically for firearm recognition. YOLO models are designed to detect objects in real time through inference, allowing efficient object localization, identification, and classification. Their architecture allows a great balance between speed and detection accuracy, making them well suited for time sensitive applications such as *EdgeSafe*. Rather than designing a new neural network architecture, the system will integrate this existing architecture for firearm/weapon detection. Prior research already supports the viability of handgun detection using deep learning [6], supporting the use of YOLO-based approach for this project.

5.2.2 Edge Deployment Platform

EdgeSafe will be designed for the deployment on low-power embedded devices such as the NVIDIA Jetson Nano or the Raspberry Pi, which are frequently used in edge AI research projects because they permit on device inference without relying on cloud infrastructure [15]. Running inference directly on the device lowers response time and enhances privacy by keeping sensitive visual data within the local system.

5.2.3 Backend and Web Frameworks

The system shall have the edge device that will host both the machine learning model and also the backend service responsible for processing detections and logging results. The communication between the frontend and backend will be handled through RESTful API principles using well structured JSON data exchange to make sure data transmission is organized. The dashboard interface will be developed using React, a frontend library that supports maintainable UI design [7]. React will allow the frontend part of the project to have the detection results, logged events and anything that needs to be displayed to be done so in a responsive single page application.

5.2.4 Local Database and Secure Logging

Events that are detected will be stored using SQLite, an ACID-compliant database system [8]. SQLite is a well suited choice for environments with limited hardware resources because it operates as a serverless database that stores data locally on the edge device. In this case, the database will store detection metadata as it has been mentioned before, such as timestamps, confidence scores, image reference, and more. Communication between client and server will be always secured using HTTPS to ensure encrypted transactions of sensitive information and data. By using SQLite for logging and HTTPS-secured communication (TLS encryption), the system will ensure reliable event storage while protecting sensitive data during transmission.

5.3 Engineering Constraints

The *EdgeSafe* system is expected to operate in a safety sensitive educational environment under realistic technical and operational limitations. This section defines the quantitative and qualitative engineering constraints that the system must satisfy. These constraints directly affect and influence the architectural decisions, hardware and software design and selection, and the deployment strategy. The constraints are categorized into: performance, hardware and resource limitations, privacy and security, and accessibility.

5.3.1 Performance Constraints

The system must satisfy performance requirements to ensure its practical viability in such a critical environment such as educational institutions. The most critical performance constraints concern inference latency. The system shall achieve an end to end inference time of one second or less per image under normal conditions. This means, the total elapsed time between image capture/upload, model inference execution and the display of the detection results to the dashboard. This constraint is essential because delayed threat recognition reduces the system's usefulness in real life scenarios and situations. In security sensitive contexts, even the smallest delays might compromise safe intervention in a timely manner. To satisfy this requirement, the system will deploy a lightweight object detection architecture for embedded hardware. Image resolution is carefully selected to balance detection reliability and computational speed, and inference is executed on edge devices to eliminate cloud transmission latency. Performance will be measured experimentally and reported as an average inference time across multiple trials to ensure reproducibility.

In addition to latency, detection accuracy represents a fundamental performance constraint. The deployed weapon detection model must achieve a minimum accuracy of 80% on a representative test dataset aligned with the target application domain. Performance evaluation will include accuracy metrics, precision, and F1 score to ensure good and acceptable performance. This will also help minimize and identify possible false negatives and false positives for the detection of weapons. These metrics are important and necessary

because too many false alarms may cause user distrust, and missed detections can undermine system credibility.

5.3.2 Hardware and Resource Constraints

The proposed system must operate within the computational limitations of edge devices such as the Raspberry Pi or the NVIDIA Jetson Nano. These devices mean that there will be constraints on memory, processing power, storage capacity, and more. The system shall operate within the memory limitations of commonly available edge devices (e.g., 2–4 GB RAM configurations), where the operating system, web server, inference engine, and logging components share the same hardware resources. As a result of this, model selection and architectural decisions will be carefully balanced to ensure system stability and reliability.

5.3.3 Privacy and Security Constraints

Because *EdgeSafe* is intended for its deployment in educational settings, this means that data protection and privacy are critical engineering constraints. The system must ensure that sensitive visual data remains within the controlled institutional environment and is accessible only to authorized personnel. All of the communication between the web client and the edge device should be encrypted to guarantee confidentiality and integrity of the data. Authentication will be required for all dashboard access, and user credentials must be securely stored. Additionally, the system follows a data minimization principle by storing only essential event metadata. This includes timestamps, detection confidence scores, and image references necessary. These measures reduce the risk of data breaches and align with responsible AI deployment practices in environments involving protected data.

5.3.4 Accessibility Constraints

The system must be accessible and usable to users with different levels of technical experience. Since the system is intended for educational environments and situations, it is important that users can easily understand and interact with the interface. The user interface will prioritize clarity and simplicity. Detection results and logged events should be presented in a clear and readable way, avoiding unnecessary complexity. Visual alerts should not only rely on color, and text should have sufficient contrast with the background to ensure readability. Navigation should strive to be intuitive enough so that users can quickly access relevant information without confusion.

5.4 Engineering Standards

The design of *EdgeSafe* will adhere to current established engineering standards in ethics, security, communication, and project management standards to ensure responsible and organized implementation of the project. From an ethical standpoint, the project will align with the IEEE Code of Ethics [10], a code that emphasizes responsibility, protection of public

safety, respect of the privacy of others, and more. Because EdgeSafe will be used in educational settings that will process potentially sensitive visual data, data protection will be taken into account from an ethical point of view, where data protection and transparency will be fundamental for design decisions. From a security point of view, the system will reflect principles highlighted in NIST Cybersecurity Framework [11], where guidance for identifying possible risks and protecting data that is sensitive is provided. The system will include secure handling of detection logs, encrypted communication, and controlled access. Communication between the system components will comply with the HTTP and HTTPS standards as defined by the IETF [13]. Encryption transportation will be provided through Transport Layer Security (TLS), as said in RFC 8446 [12], ensuring confidentiality of the transmitted data between the client interface and the chosen edge device. Finally, in terms of project management for the semester project, the team will follow the Agile principles for the development methodology [9]. This approach emphasizes iterative development, adaptability to evolving new requirements, and continuous improvement. Constantly refining the system components throughout the project lifecycle.

5.5 Best Practices

The team will adopt an iterative development approach aligned with Scrum principles, including clearly defined tasks, short development cycles, and regular progress evaluation [14]. Weekly team meetings will be conducted to review progress, address technical challenges, and plan upcoming work. Source control will be managed using Git and hosted on GitHub to enable version tracking, collaborative development, and safe integration of system components. All major code contributions will undergo peer code review prior to integration to improve reliability, maintain coding standards, and reduce defects. Together, these practices support transparent project management, reproducible development, and maintainable system design consistent with modern software engineering standards.

6. Project Plan

6.1 Development Process and Strategy

The development of EdgeSafe will follow an iterative methodology aligned with Agile and Scrum principles as described before. The approach to be taken in this project will involve a division of labor in teams of two, where one team will be focused on the YOLO-based model's optimization and integration into the edge device and the other will be focused on the implementation of the backend API, database, logging system, authentication, and security. Each team will be working in parallel in order to cut the time required to complete the project, leaving a significant amount of time left to integrate and test each team's part of the project. This does not mean that each team is not responsible for the other team's success in their part of the project. There will be collaboration and help offered if a team needs assistance from the other team in their part of the project. Additionally, there are

tasks in which each team member has to work on together. These tasks include documentation, architecture design, testing, validation, and the presentation of the final demo. Weekly team meetings will be conducted to review progress, reassess risks, and refine implementation priorities in response to technical findings.

6.2 Timetable of Milestones and Deliverables

Milestone and Deliverables	**Description**	**Team Assigned to Task**	**Date**
Edge Device Setup	Configure edge device environment.	Andrea Seda Jessy Andujar	March 9, 2026
Backend API and Secure Event Logging	Implement structured event logging and secure credential schemas as well as a local database and REST API.	Jorge Marín Marymar Maldonado	March 16, 2026
YOLO-Based Model Optimization and Setup	Find, setup, and optimize (compress) YOLO-based firearm detection model.	Andrea Seda Jessy Andujar	March 23, 2026
Test Model Functionality and Logging Integration	Test, improve, and benchmark the YOLO-based model's accuracy at firearm detection as well as logging response and storage in the database.	Everyone	April 6. 2026
Progress Report	Submit a progress report document where we address reviews in this proposal, updated design elements, discussion of preliminary	Everyone	April 16, 2026

	performance results, and project documentation.		
Dashboard and Authentication	Develop a web-based dashboard with an authentication system that connects to the REST API and displays metadata about possible firearm detections.	Jorge Marín Marymar Maldonado	April 27, 2026
Final Testing	Test and finish development of the project.	Everyone	May 4, 2026
Final Report	Submit a final report with an update of design elements and performance results.	Everyone	May 18, 2026
Final Demo	Final presentation and demo	Everyone	May 18, 2026

6.3 Testing Plan

In order to ensure compliance with the defined performance objectives and engineering constraints, each part of EdgeSafe will follow a structured and layered testing strategy. The system will be validated progressively; will be tested in isolation first in order to catch any existing bugs or problems that would need to be addressed at the time of testing advancing toward full end-to-end evaluation. This helps us mitigate the amount of debugging required when testing EdgeSafe as a whole after making sure each part of the project works as expected beforehand.

1. Unit Testing:

   Each subsystem will be tested independently to ensure correct functionality prior to full integration:

   - Inference Module
     - Verify correct output format (object labels and confidence scores)
     - Confirm consistent detection behavior across repeated trials
     - Validate compatibility with the defined JSON response structure

- Logging and Database Module
    - Confirm that detection events are correctly stored in the detection_events table
    - Validate referential integrity between event metadata and associated image artifacts
    - Ensure successful retrieval of stored events via API
- Authentication Mechanism
    - Confirm protected routes require valid credentials
    - Attempt unauthorized access to ensure proper rejection
- API Endpoints
    - Validate correct HTTP status codes
    - Confirm structured JSON responses
    - Test error handling behavior

2. Performance Testing

   To validate compliance with system performance objectives:

   - Latency Testing
     - Measure end-to-end inference latency
     - Confirm average inference time ≤ 1 second per image
     - Report results across multiple trials for reproducibility
   - Detection Accuracy Evaluation
     - Evaluate model performance on a representative test dataset
     - Compute accuracy, precision, recall, and F1-score
     - Confirm minimum detection accuracy ≥ 80%

   These metrics ensure that the system satisfies both latency and detection reliability requirements.

3. Integration Testing

   Full pipeline testing will be conducted after subsystem validation:

   Image input -> Inference -> Event generation -> Database storage -> API retrieval -> Dashboard display

   The objective is to confirm seamless interaction between components without data loss or corruption.

4. Reliability and Stability Testing

   To ensure system robustness under realistic conditions:

   - Conduct extended runtime tests on the edge device
   - Monitor CPU and memory utilization

- Verify no system crashes occur during continuous execution
- Confirm all logged events remain retrievable after prolonged operation

5. Traceability and Logging Integrity Validation

   To satisfy accountability and auditability requirements:

   - Verify that every detected event produces a corresponding database record
   - Confirm timestamp accuracy and metadata completeness
   - Validate that logged events remain immutable and consistently retrievable

7. Previous Work and References:

[1] J. K. Peterson, J. A. Densley, M. Hauf, and J. Moldenhauer, "Epidemiology of Mass Shootings in the United States," *Annu. Rev. Clin. Psychol.*, vol. 20, pp. 125–148, Jul. 2024, doi: 10.1146/annurev-clinpsy-081122-010256.

[2] Lee LK, Fleegler EW, Goyal MK, Doh KF, Laraque-Arena D, Hoffman BD, Injury Violence And Poison Prevention CO. Firearm-Related Injuries and Deaths in Children and Youth. Pediatrics. 2022 Oct 8. doi: 10.1542/peds.2022-060071. Epub ahead of print. PMID: 36207778.

[3] J. Redmon et al., "You Only Look Once: Unified, Real-Time Object Detection," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

[4] W. Shi et al., "Edge Computing: Vision and Challenges," *IEEE Internet of Things Journal*, 2020. https://cse.buffalo.edu/faculty/tkosar/cse710_spring20/shi-iot16.pdf

[5] R. Mitchell et al., "Auditability and Accountability in AI-Based Surveillance Systems," *ACM Conference on Computer and Communications Security (CCS)*, 2022. https://facctconference.org/static/pdfs_2022/facct22-3533150.pdf

[6] R. Olmos, S. Tabik, and F. Herrera, "Automatic handgun detection alarm in videos using deep learning," *Neurocomputing,* vol. 275, pp. 66-72, 2018. https://www.sciencedirect.com/science/article/pii/S0925231217308196

[7] React, "Describing the UI," *React Documentation*. https://react.dev/learn/describing-the-ui

[8] SQLite, "SQLite is Transactional," *SQLite Documentation.* https://www.sqlite.org/transactional.html

[9] Agile Alliance, "Agile 1010, What is Agile?," *Agile Alliance*. https://agilealliance.org/agile101/

[10] IEEE, "IEEE Code of Ethics," *IEEE,* 2020. https://www.ieee.org/about/corporate/governance/p7-8

[11] NIST, “Framework for Improving Critical Infrastructure Cybersecurity,” *National Institute of Standards and Technology,* 2018. https://www.nist.gov/cyberframework

[12] E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” *RFC 8446, Internet Engineering Task Force (IETF), Aug. 2018.* https://datatracker.ietf.org/doc/html/rfc8446

[13] R. Fielding et al., “Hypertext Transfer Protocol - HTTP /1.1,” RFC 2616, *Internet Engineering Task Force (IETF)*, Jun 1999. https://datatracker.ietf.org/doc/html/rfc2616

[14] A. Srivastava, S. Bhardwaj, and S. Saraswat, “SCRUM Model for Agile Methodology,” in Proc. Int. Conf. Computing, Communication and Automation (ICCCA), 2017, pp. 864–869.

[15] M. Satyanarayanan, “The Emergence of Edge Computing,” *Computer*, vol. 50 no. 1, pp. 30-39, 2017. doi: 10.1109/MC.2017.9.